

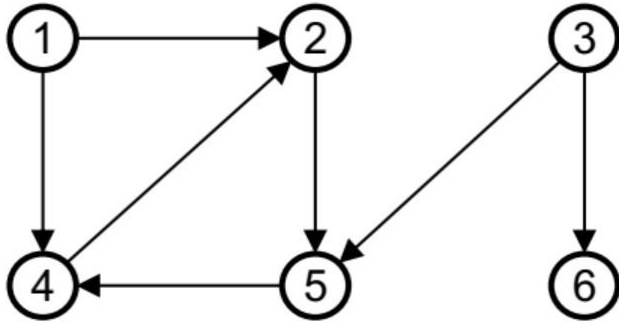
Data Structures

Lecture 21: Graph Traversal

Soobin Um

Jun. 1, 2026

Recap: Definition of Graph



$$V = \{1,2,3,4,5,6\}$$

$$E = \{(1,2), (1,4), (2,5), (3,5), (3,6), (4,2), (5,4)\}$$

- 그래프 $G=(V, E)$ 는 다음의 두 가지 집합으로 이루어진 자료 구조
 - 정점 집합 V (Vertices)
 - 간선 집합 E (Edges)
- 정점의 개수는 $|V|$, 간선의 개수는 $|E|$ 로 표기

Graph Traversal (그래프 순회)

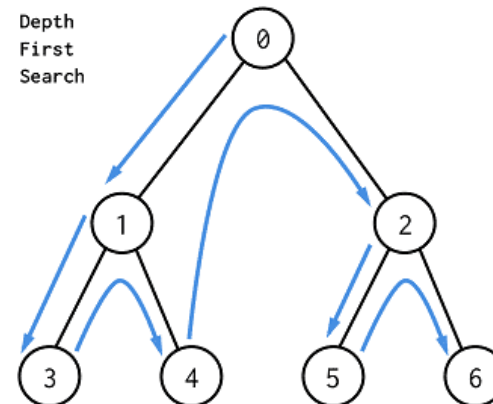
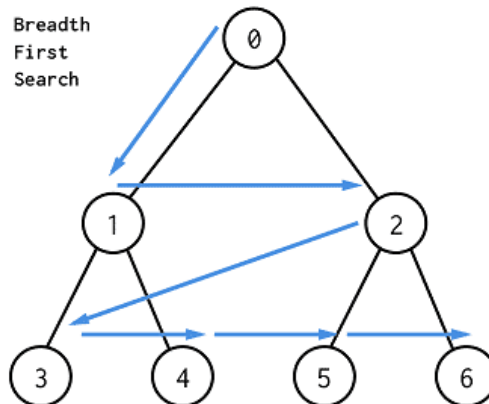
- 존재하는 모든 정점을 정확히 한 번씩 방문하는 절차

Graph Traversal (그래프 순회)

- 존재하는 모든 정점을 정확히 한 번씩 방문하는 절차
- 응용분야
 - 최단 경로 문제: 정점 간 이동 비용이 주어졌을 때, 가장 짧은 (최소 비용) 경로를 찾기
 - 연결 요소 찾기: 그래프의 각 부분이 하나의 연결된 “덩어리”인지를 판별
 - 네트워크 분석: 네트워크 상 영향력이 특정 정점에서 시작해 어떻게 확산되는지를 식별

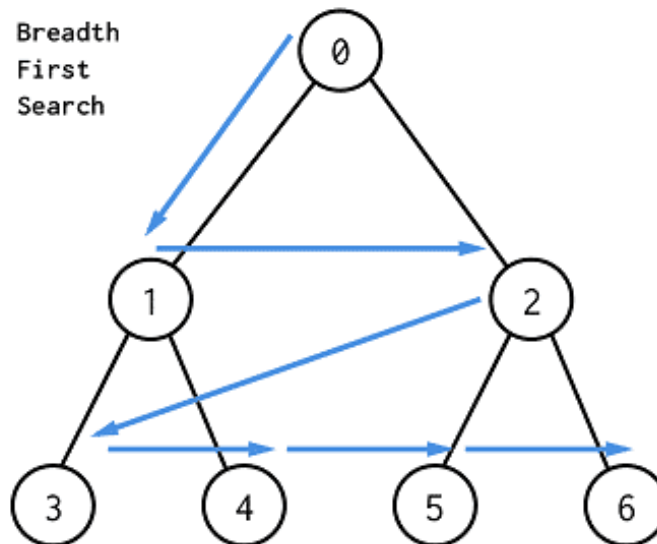
Graph Traversal (그래프 순회)

- 존재하는 모든 정점을 정확히 한 번씩 방문하는 절차
- 응용분야
 - 최단 경로 문제: 정점 간 이동 비용이 주어졌을 때, 가장 짧은 (최소 비용) 경로를 찾기
 - 연결 요소 찾기: 그래프의 각 부분이 하나의 연결된 “덩어리”인지를 판별
 - 네트워크 분석: 네트워크 상 영향력이 특정 정점에서 시작해 어떻게 확산되는지를 식별
- 주요 그래프 순회 기법
 - 너비 우선 탐색 (Breadth-First Search, BFS)
 - 깊이 우선 탐색 (Depth-First Search, DFS)



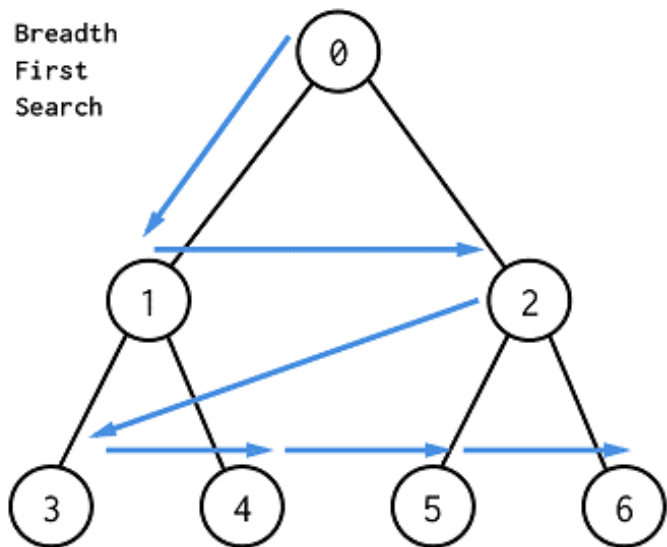
Breadth-First Search (너비 우선 탐색)

- 현재 정점에서 가까운 정점부터 먼 정점의 순서로 (레벨 별) 탐색을 진행



Breadth-First Search (너비 우선 탐색)

- 현재 정점에서 **가까운 정점부터 먼 정점의 순서**로 (레벨 별) 탐색을 진행
- 핵심 아이디어: **거리 기반 탐색**
 - 시작 정점으로부터의 거리(최단 경로 상 간선 수)가 작은 순서대로 정점들을 방문
 - 1단계 (레벨): 거리 1에 있는 정점들을 모두 방문
 - 2단계: 거리 2에 있는 정점들을 모두 방문
 - ...
 - 모든 정점들이 방문될 때까지 이러한 과정을 반복
 - 큐 자료 구조를 이용

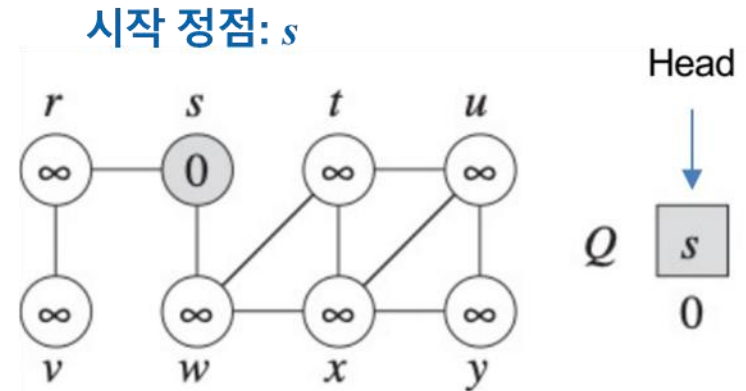


알고리즘: BFS

BFS(G, s)

/* 초기화 */

```
1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4   $s.color = GRAY$ 
5   $s.d = 0$ 
6   $Q = \emptyset$ 
7  Enqueue( $Q, s$ )
```



- **White:** 아직 방문 되지 않음 (enqueue)
- **Gray:** 방문 되었지만 아직 완료되진 않음
- **Black:** 완료 (dequeue)

알고리즘: BFS

BFS(G, s)

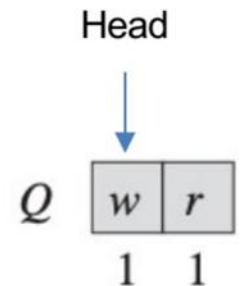
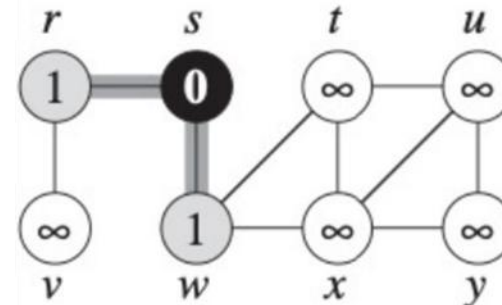
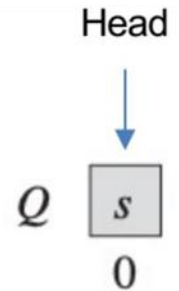
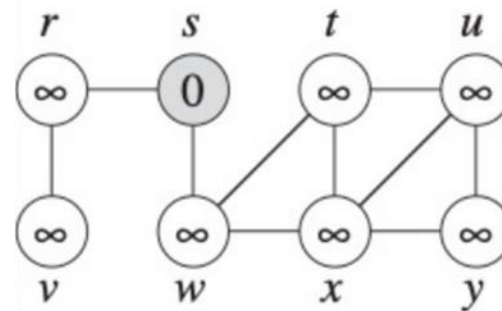
/* 초기화 */

```
1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4   $s.color = GRAY$ 
5   $s.d = 0$ 
6   $Q = \emptyset$ 
7  Enqueue( $Q, s$ )
```

/* BFS */

```
8  while  $Q \neq \emptyset$ 
9     $u = \text{Dequeue}(Q)$ 
10   for each  $v \in G.Adj[u]$  // 이웃 정점
11     if  $v.color == WHITE$ 
12        $v.color = GRAY$ 
13        $v.d = u.d + 1$ 
14       Enqueue( $Q, v$ )
15    $u.color = BLACK$ 
```

시작 정점: s



- **White:** 아직 방문 되지 않음 (enqueue)
- **Gray:** 방문 되었지만 아직 완료되진 않음
- **Black:** 완료 (dequeue)

알고리즘: BFS

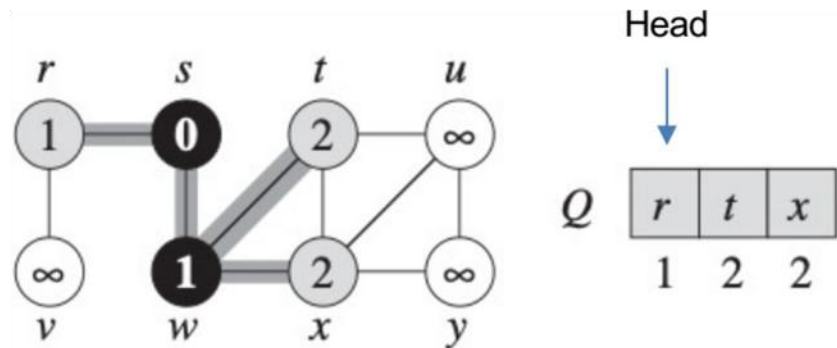
BFS(G, s)

/* 초기화 */

```
1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4   $s.color = GRAY$ 
5   $s.d = 0$ 
6   $Q = \emptyset$ 
7  Enqueue( $Q, s$ )
```

/* BFS */

```
8  while  $Q \neq \emptyset$ 
9     $u = Dequeue(Q)$ 
10   for each  $v \in G.Adj[u]$  // 이웃 정점
11     if  $v.color == WHITE$ 
12        $v.color = GRAY$ 
13        $v.d = u.d + 1$ 
14       Enqueue( $Q, v$ )
15    $u.color = BLACK$ 
```



- **White:** 아직 방문 되지 않음 (enqueue)
- **Gray:** 방문 되었지만 아직 완료되지 않음
- **Black:** 완료 (dequeue)

알고리즘: BFS

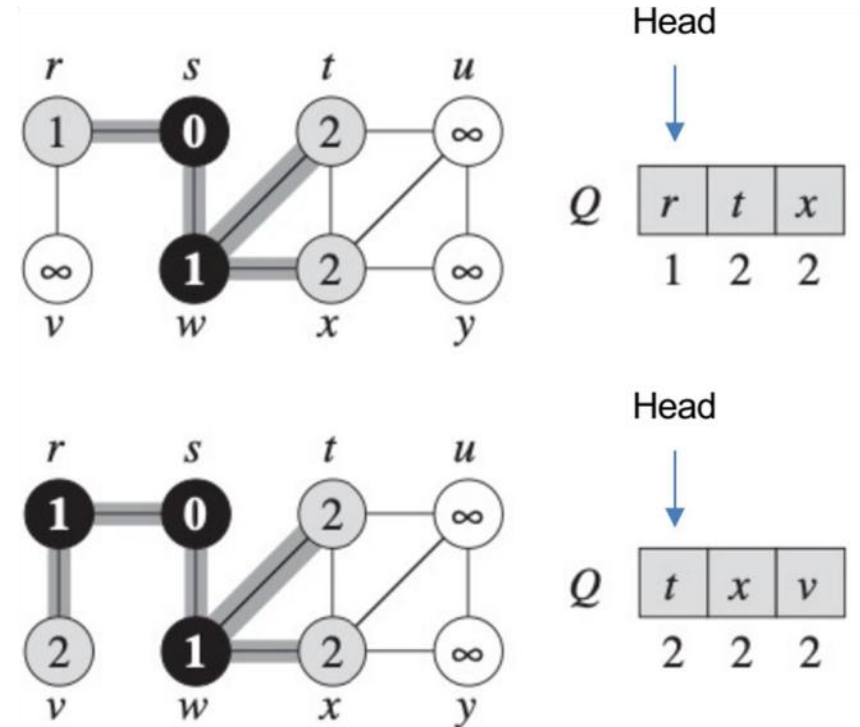
BFS(G, s)

/* 초기화 */

```
1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4   $s.color = GRAY$ 
5   $s.d = 0$ 
6   $Q = \emptyset$ 
7  Enqueue( $Q, s$ )
```

/* BFS */

```
8  while  $Q \neq \emptyset$ 
9     $u = \text{Dequeue}(Q)$ 
10   for each  $v \in G.Adj[u]$  // 이웃 정점
11     if  $v.color == WHITE$ 
12        $v.color = GRAY$ 
13        $v.d = u.d + 1$ 
14       Enqueue( $Q, v$ )
15    $u.color = BLACK$ 
```



- **White:** 아직 방문 되지 않음 (enqueue)
- **Gray:** 방문 되었지만 아직 완료되진 않음
- **Black:** 완료 (dequeue)

알고리즘: BFS

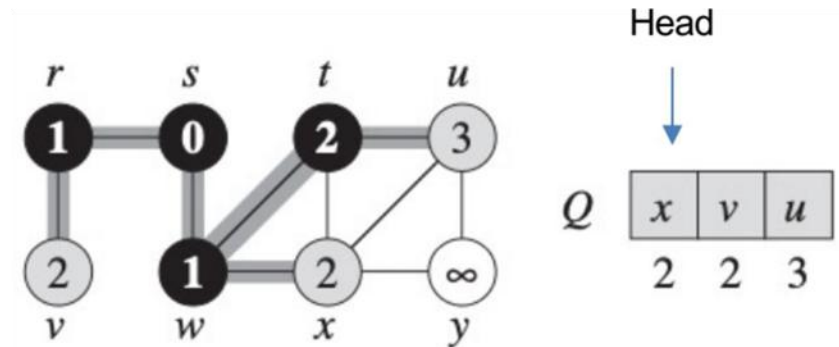
BFS(G, s)

/* 초기화 */

```
1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4   $s.color = GRAY$ 
5   $s.d = 0$ 
6   $Q = \emptyset$ 
7  Enqueue( $Q, s$ )
```

/* BFS */

```
8  while  $Q \neq \emptyset$ 
9     $u = \text{Dequeue}(Q)$ 
10   for each  $v \in G.Adj[u]$  // 이웃 정점
11     if  $v.color == WHITE$ 
12        $v.color = GRAY$ 
13        $v.d = u.d + 1$ 
14       Enqueue( $Q, v$ )
15    $u.color = BLACK$ 
```



- **White:** 아직 방문 되지 않음 (enqueue)
- **Gray:** 방문 되었지만 아직 완료되지 않음
- **Black:** 완료 (dequeue)

알고리즘: BFS

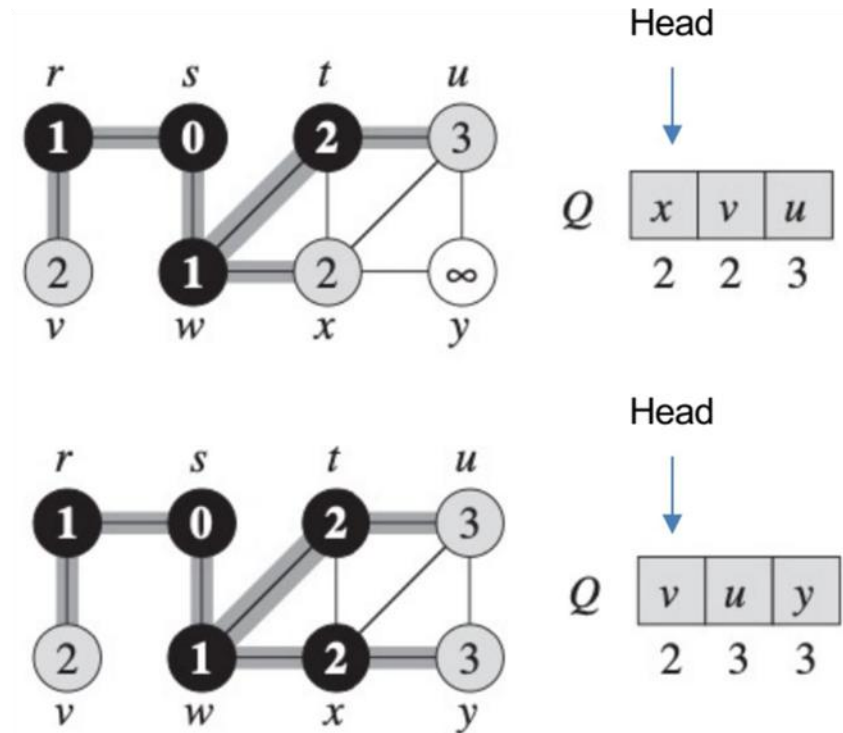
BFS(G, s)

/* 초기화 */

```
1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4   $s.color = GRAY$ 
5   $s.d = 0$ 
6   $Q = \emptyset$ 
7  Enqueue( $Q, s$ )
```

/* BFS */

```
8  while  $Q \neq \emptyset$ 
9     $u = \text{Dequeue}(Q)$ 
10   for each  $v \in G.Adj[u]$  // 이웃 정점
11     if  $v.color == WHITE$ 
12        $v.color = GRAY$ 
13        $v.d = u.d + 1$ 
14       Enqueue( $Q, v$ )
15    $u.color = BLACK$ 
```



- **White:** 아직 방문 되지 않음 (enqueue)
- **Gray:** 방문 되었지만 아직 완료되진 않음
- **Black:** 완료 (dequeue)

알고리즘: BFS

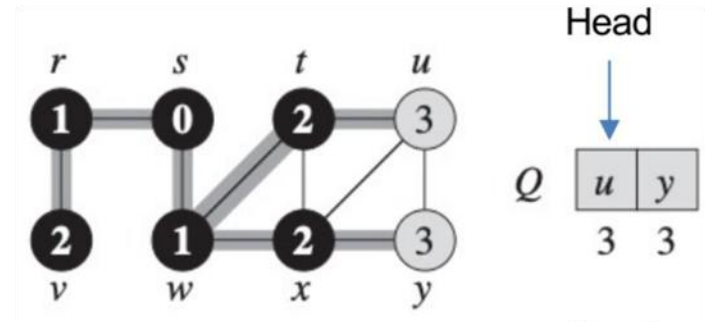
BFS(G, s)

/* 초기화 */

```
1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4   $s.color = GRAY$ 
5   $s.d = 0$ 
6   $Q = \emptyset$ 
7  Enqueue( $Q, s$ )
```

/* BFS */

```
8  while  $Q \neq \emptyset$ 
9     $u = Dequeue(Q)$ 
10   for each  $v \in G.Adj[u]$  // 이웃 정점
11     if  $v.color == WHITE$ 
12        $v.color = GRAY$ 
13        $v.d = u.d + 1$ 
14       Enqueue( $Q, v$ )
15    $u.color = BLACK$ 
```



- **White:** 아직 방문 되지 않음 (enqueue)
- **Gray:** 방문 되었지만 아직 완료되진 않음
- **Black:** 완료 (dequeue)

알고리즘: BFS

BFS(G, s)

/* 초기화 */

```

1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4   $s.color = GRAY$ 
5   $s.d = 0$ 
6   $Q = \emptyset$ 
7  Enqueue( $Q, s$ )

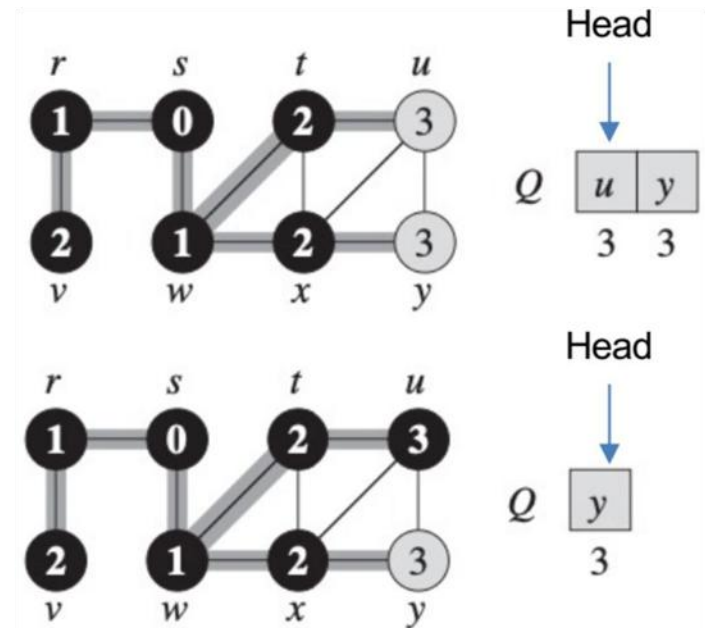
```

/* BFS */

```

8  while  $Q \neq \emptyset$ 
9     $u = \text{Dequeue}(Q)$ 
10   for each  $v \in G.Adj[u]$  // 이웃 정점
11     if  $v.color == WHITE$ 
12        $v.color = GRAY$ 
13        $v.d = u.d + 1$ 
14       Enqueue( $Q, v$ )
15    $u.color = BLACK$ 

```



- **White:** 아직 방문 되지 않음 (enqueue)
- **Gray:** 방문 되었지만 아직 완료되진 않음
- **Black:** 완료 (dequeue)

알고리즘: BFS

BFS(G, s)

/* 초기화 */

```

1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4   $s.color = GRAY$ 
5   $s.d = 0$ 
6   $Q = \emptyset$ 
7  Enqueue( $Q, s$ )

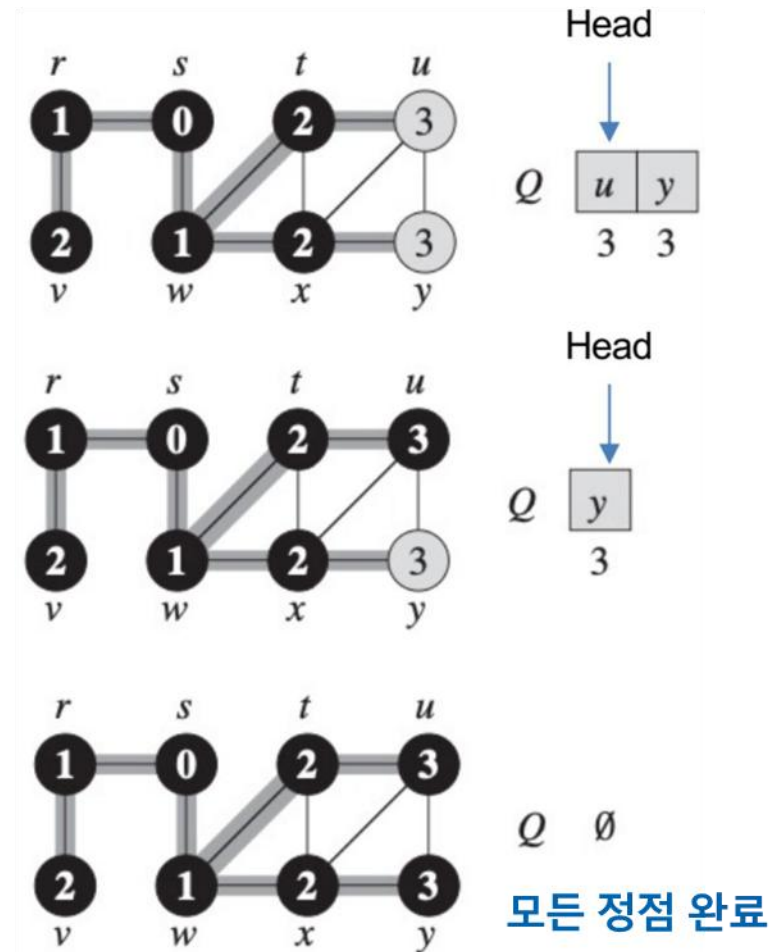
```

/* BFS */

```

8  while  $Q \neq \emptyset$ 
9     $u = Dequeue(Q)$ 
10   for each  $v \in G.Adj[u]$  // 이웃 정점
11     if  $v.color == WHITE$ 
12        $v.color = GRAY$ 
13        $v.d = u.d + 1$ 
14       Enqueue( $Q, v$ )
15    $u.color = BLACK$ 

```



- **White:** 아직 방문 되지 않음 (enqueue)
- **Gray:** 방문 되었지만 아직 완료되진 않음
- **Black:** 완료 (dequeue)

시간 복잡도: BFS

- 초기화: $O(|V|)$
 - 모든 정점의 색 초기화
 - 시작 정점으로부터의 거리 초기화

BFS(G, s)

/ 초기화 */*

```
1  for each vertex  $u \in G.V - \{s\}$ 
2       $u.color = WHITE$ 
3       $u.d = \infty$ 
4   $s.color = GRAY$ 
5   $s.d = 0$ 
6   $Q = \emptyset$ 
7  Enqueue( $Q, s$ )
```

시간 복잡도: BFS

- 초기화: $O(|V|)$
 - 모든 정점의 색 초기화
 - 시작 정점으로부터의 거리 초기화
- 그래프 순회 (방문): $O(|V|+|E|)$
 - 각 정점은 정확히 한 번씩만 방문: $O(|V|)$
 - 각 간선에 대한 방문
 - 무향 그래프: 간선 하나가 양쪽 정점의 리스트에 나타남: $O(2|E|) = O(|E|)$
 - 유향 그래프: 간선마다 정확히 한 번씩만 방문: $O(|E|)$

BFS(G, s)

/* 초기화 */

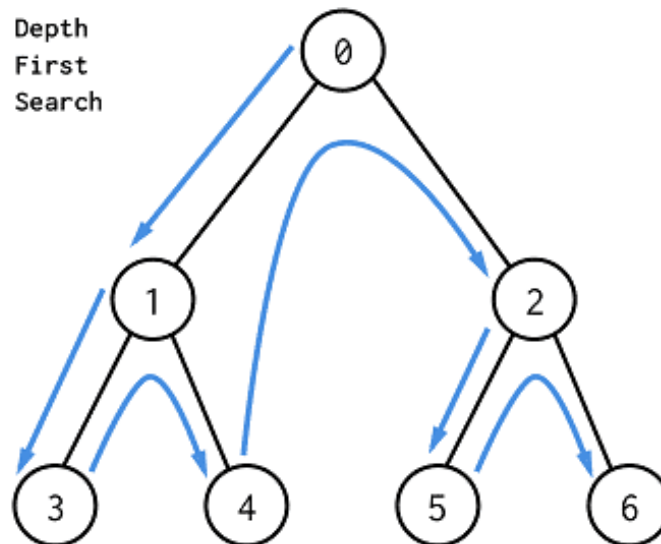
```
1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4   $s.color = GRAY$ 
5   $s.d = 0$ 
6   $Q = \emptyset$ 
7  Enqueue( $Q, s$ )
```

/* BFS */

```
8  while  $Q \neq \emptyset$ 
9     $u = Dequeue(Q)$ 
10   for each  $v \in G.Adj[u]$  // 이웃 정점
11     if  $v.color == WHITE$ 
12        $v.color = GRAY$ 
13        $v.d = u.d + 1$ 
14       Enqueue( $Q, v$ )
15    $u.color = BLACK$ 
```

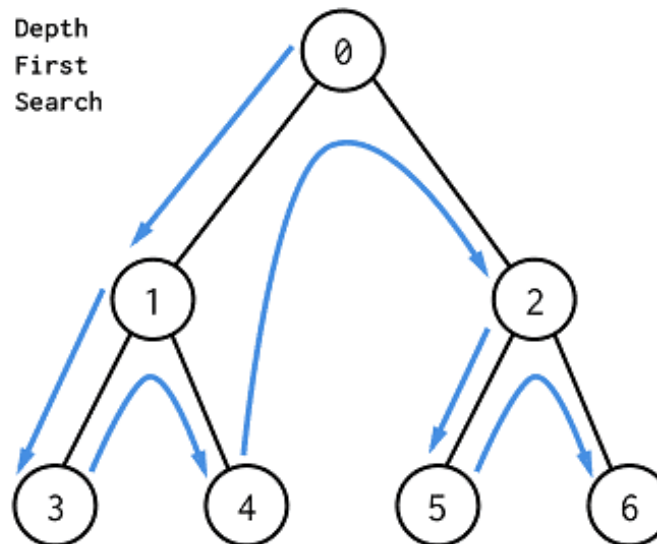
Depth-First Search (깊이 우선 탐색)

- 가지를 따라 가능한 한 깊숙이 내려간 뒤, 더 이상 갈 곳이 없으면 되돌아 올



Depth-First Search (깊이 우선 탐색)

- 가지를 따라 가능한 한 깊숙이 내려간 뒤, 더 이상 갈 곳이 없으면 되돌아 오
- 핵심 아이디어: **깊이 기반 탐색**
 - 현재 정점에서 갈 수 있는 정점이 있다면 계속 깊이 들어가며 탐색
 - 새로운 이웃을 더 이상 찾을 수 없을 경우, 이전 정점으로 되돌아 감
 - Backtracking을 기반으로 한 **재귀적 구조**를 지님



알고리즘: DFS

DFS(G)

/* 초기화 */

```
1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
```

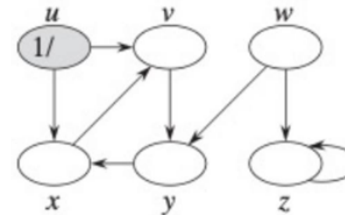
알고리즘: DFS

DFS(G)

/* 초기화 */

```
1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
```

(1)



DFS-Visit(G, u)

```
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
```

정점 발견 (d)

깊이 내려가며
이웃 탐색

되돌아 온 뒤
완료 (f)

알고리즘: DFS

DFS(G)

/ 초기화 */*

```
1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
```

DFS-Visit(G, u)

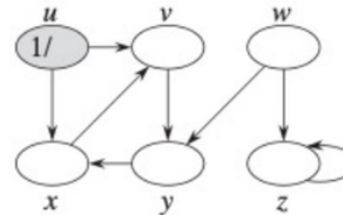
```
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
```

정점 발견 (d)

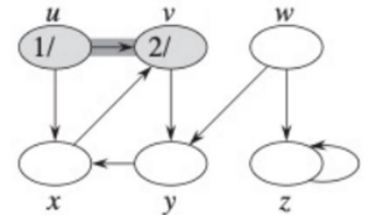
깊이 내려가며
이웃 탐색

되돌아 온 뒤
완료 (f)

(1)



(2)



알고리즘: DFS

DFS(G)

/* 초기화 */

```
1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
```

DFS-Visit(G, u)

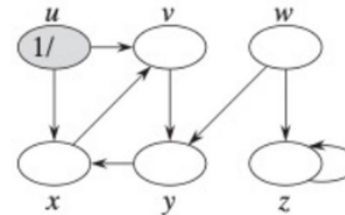
```
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
```

정점 발견 (d)

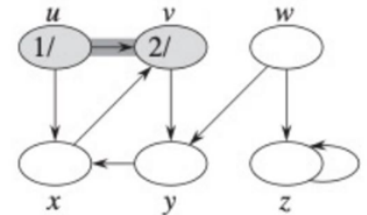
깊이 내려가며
이웃 탐색

되돌아 온 뒤
완료 (f)

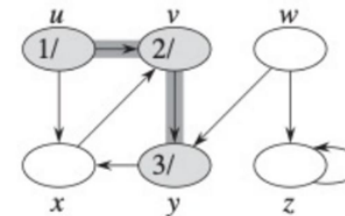
(1)



(2)



(3)



알고리즘: DFS

DFS(G)

/* 초기화 */

```
1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
```

DFS-Visit(G, u)

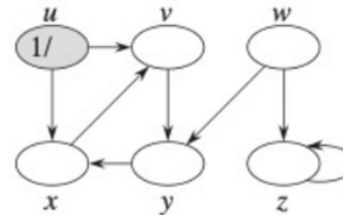
```
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
```

정점 발견 (d)

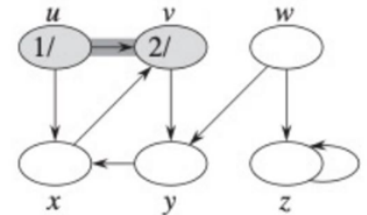
깊이 내려가며
이웃 탐색

되돌아 온 뒤
완료 (f)

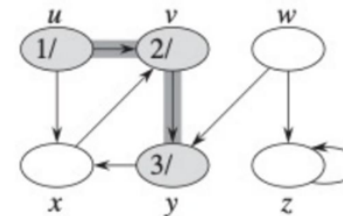
(1)



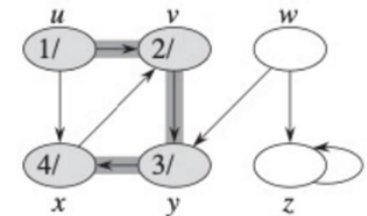
(2)



(3)



(4)



알고리즘: DFS

DFS(G)

/* 초기화 */

```
1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
```

DFS-Visit(G, u)

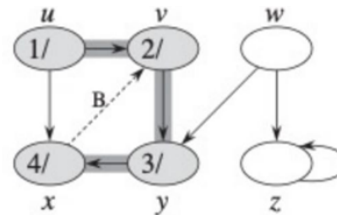
```
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
```

정점 발견 (d)

깊이 내려가며
이웃 탐색

되돌아 온 뒤
완료 (f)

(1)



- B: back edge (정점에서 조상으로 향하는 간선)

알고리즘: DFS

DFS(G)

/* 초기화 */

```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
    
```

DFS-Visit(G, u)

```

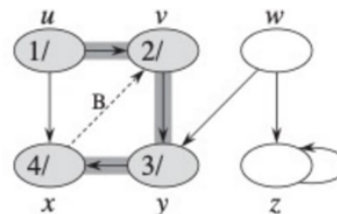
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
    
```

정점 발견 (d)

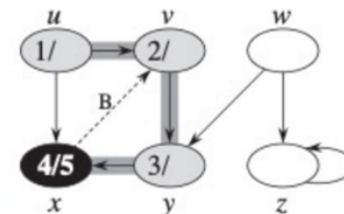
깊이 내려가며 이웃 탐색

되돌아 온 뒤 완료 (f)

(1)



(2)



- B: back edge (정점에서 조상으로 향하는 간선)

알고리즘: DFS

DFS(G)

/* 초기화 */

```
1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
```

DFS-Visit(G, u)

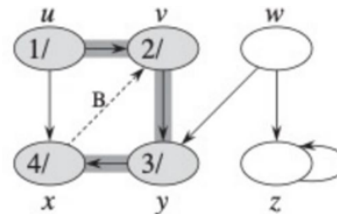
```
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
```

정점 발견 (d)

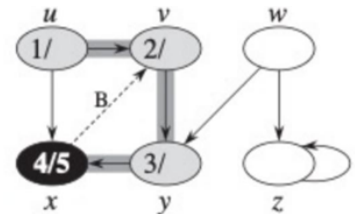
깊이 내려가며 이웃 탐색

되돌아 온 뒤 완료 (f)

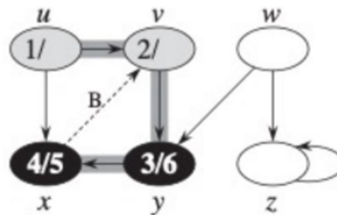
(1)



(2)



(3)



- B: back edge (정점에서 조상으로 향하는 간선)

알고리즘: DFS

DFS(G)

/* 초기화 */

```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
    
```

DFS-Visit(G, u)

```

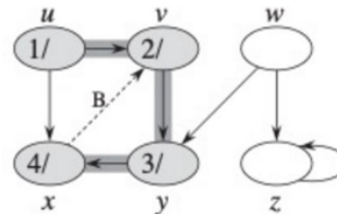
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
    
```

정점 발견 (d)

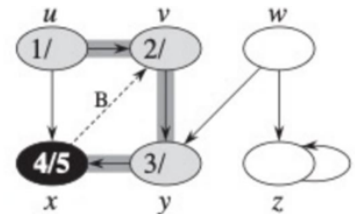
깊이 내려가며 이웃 탐색

되돌아 온 뒤 완료 (f)

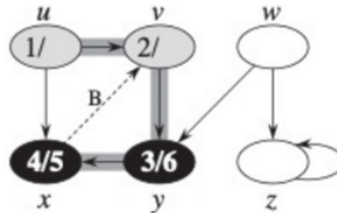
(1)



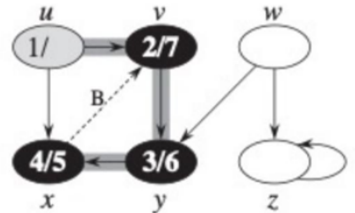
(2)



(3)



(4)



- B: back edge (정점에서 조상으로 향하는 간선)

알고리즘: DFS

DFS(G)

/* 초기화 */

```
1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
```

DFS-Visit(G, u)

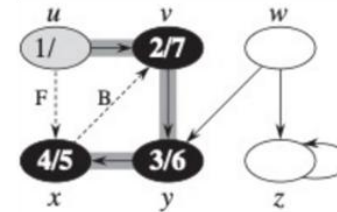
```
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
```

정점 발견 (d)

깊이 내려가며
이웃 탐색

되돌아 온 뒤
완료 (f)

(1)



- B: back edge (정점에서 조상으로 향하는 간선)
- F: forward edge (조상에서 자손으로 향하는 간선)

알고리즘: DFS

DFS(G)

/* 초기화 */

```
1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
```

DFS-Visit(G, u)

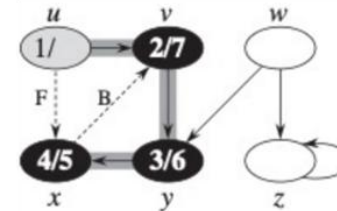
```
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
```

정점 발견 (d)

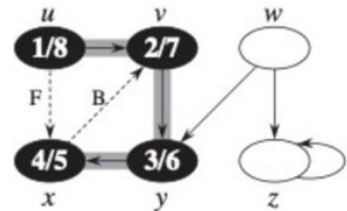
깊이 내려가며
이웃 탐색

되돌아 온 뒤
완료 (f)

(1)



(2)



- B: back edge (정점에서 조상으로 향하는 간선)
- F: forward edge (조상에서 자손으로 향하는 간선)

알고리즘: DFS

DFS(G)

/* 초기화 */

```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
    
```

DFS-Visit(G, u)

```

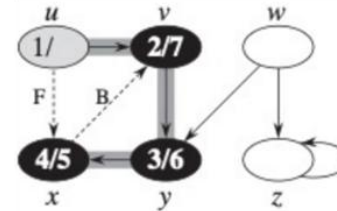
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
    
```

정점 발견 (d)

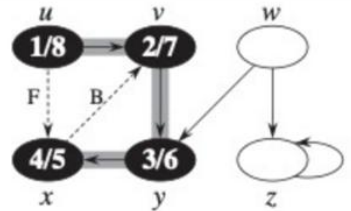
깊이 내려가며 이웃 탐색

되돌아 온 뒤 완료 (f)

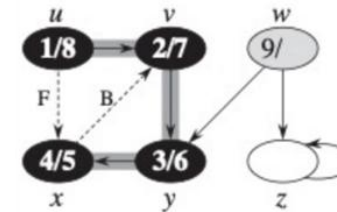
(1)



(2)



(3)



- B: back edge (정점에서 조상으로 향하는 간선)
- F: forward edge (조상에서 자손으로 향하는 간선)

알고리즘: DFS

DFS(G)

/* 초기화 */

```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
    
```

DFS-Visit(G, u)

```

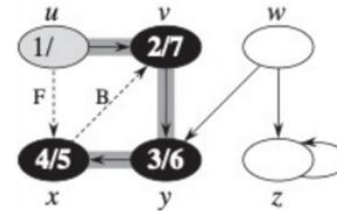
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
    
```

정점 발견 (d)

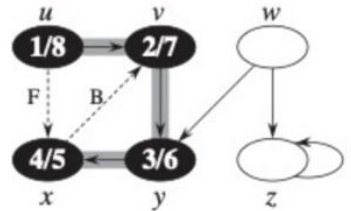
깊이 내려가며 이웃 탐색

되돌아 온 뒤 완료 (f)

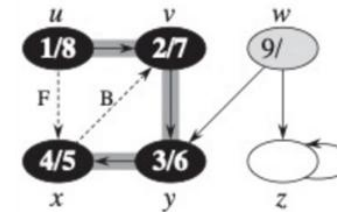
(1)



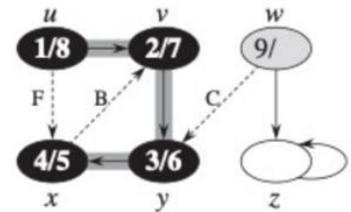
(2)



(3)



(4)



- B: back edge (정점에서 조상으로 향하는 간선)
- F: forward edge (조상에서 자손으로 향하는 간선)
- C: cross edge (서로 다른 DFS 서브 트리 간 연결)

알고리즘: DFS

DFS(G)

/* 초기화 */

```
1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
```

DFS-Visit(G, u)

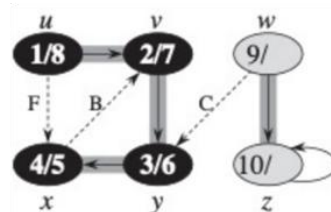
```
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
```

정점 발견 (d)

깊이 내려가며
이웃 탐색

되돌아 온 뒤
완료 (f)

(1)



- **B**: back edge (정점에서 조상으로 향하는 간선)
- **F**: forward edge (조상에서 자손으로 향하는 간선)
- **C**: cross edge (서로 다른 DFS 서브 트리 간 연결)

알고리즘: DFS

DFS(G)

/* 초기화 */

```
1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
```

DFS-Visit(G, u)

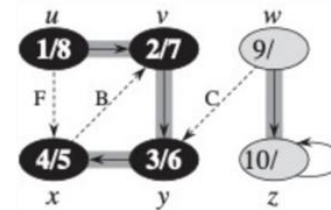
```
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
```

정점 발견 (d)

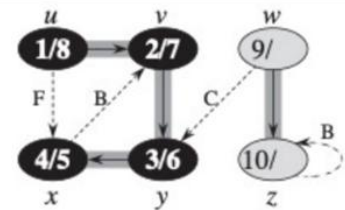
깊이 내려가며
이웃 탐색

되돌아 온 뒤
완료 (f)

(1)



(2)



- B: back edge (정점에서 조상으로 향하는 간선)
- F: forward edge (조상에서 자손으로 향하는 간선)
- C: cross edge (서로 다른 DFS 서브 트리 간 연결)

알고리즘: DFS

DFS(G)

/* 초기화 */

```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
    
```

DFS-Visit(G, u)

```

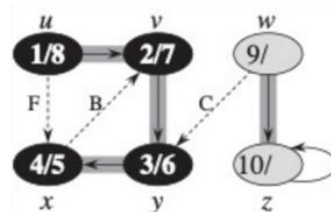
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
    
```

정점 발견 (d)

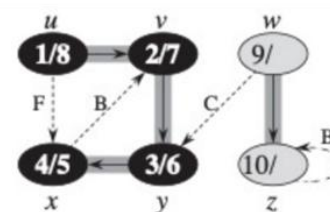
깊이 내려가며 이웃 탐색

되돌아 온 뒤 완료 (f)

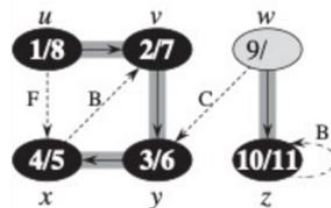
(1)



(2)



(3)



- B: back edge (정점에서 조상으로 향하는 간선)
- F: forward edge (조상에서 자손으로 향하는 간선)
- C: cross edge (서로 다른 DFS 서브 트리 간 연결)

알고리즘: DFS

DFS(G)

/* 초기화 */

```

1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
    
```

DFS-Visit(G, u)

```

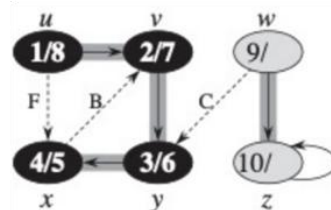
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
    
```

정점 발견 (d)

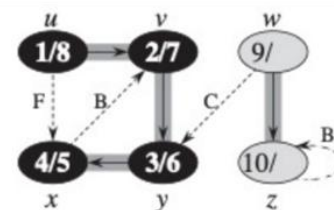
깊이 내려가며 이웃 탐색

되돌아 온 뒤 완료 (f)

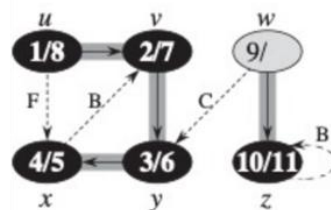
(1)



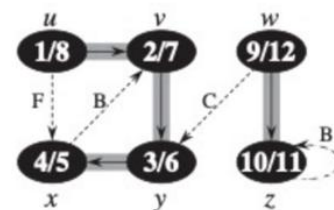
(2)



(3)



(4)



- B: back edge (정점에서 조상으로 향하는 간선)
- F: forward edge (조상에서 자손으로 향하는 간선)
- C: cross edge (서로 다른 DFS 서브 트리 간 연결)

시간 복잡도: DFS

- 초기화: $O(|V|)$
 - 모든 정점의 색 초기화
 - Discovery time 및 finish time 초기화
- 그래프 순회 (방문): $O(|V|+|E|)$
 - 각 정점은 정확히 한 번씩만 방문: $O(|V|)$
 - 각 간선에 대한 방문
 - 무향 그래프: 간선 하나가 양쪽 정점의 리스트에 나타남: $O(2|E|) = O(|E|)$
 - 유향 그래프: 간선마다 정확히 한 번씩만 방문: $O(|E|)$

DFS(G)

/ 초기화 */*

```
1  for each vertex  $u \in G.V$ 
2     $u.color = WHITE$ 
3     $u.\pi = NIL$  // 전임자 노드
4   $time = 0$ 
5  for each vertex  $u \in G.V$  // 모든 정점 대상
6    if  $u.color == WHITE$ 
7      DFS-Visit( $G, u$ )
```

DFS-Visit(G, u)

```
1   $time = time + 1$ 
2   $u.d = time$ 
3   $u.color = GRAY$ 
4  for each vertex  $v \in G.Adj[u]$ 
5    if  $v.color == WHITE$ 
6       $v.\pi = u$ 
7      DFS-Visit( $G, v$ )
8   $u.color = BLACK$ 
9   $time = time + 1$ 
10  $u.f = time$ 
```

정점 발견 (d)

깊이 내려가며 이웃 탐색

되돌아 온 뒤 완료 (f)

Look ahead

Shortest Path